Proiect de laborator

Proiectele se vor efectua in echipe de maximum 2 studenti si vor fi prezentate la laborator conform programarii stabilite cu cadrul didactic de la laborator.

La finalul proiectului, fiecare membru al echipei va declara **la care dintre punctele 1-7 de mai jos a contribuit** si **procentul propriu de contribuție la proiect**. Procentele declarate trebuie să însumeze 100%.

Utilizarea instrumentelor bazate pe inteligență artificială

Pentru elaborarea proiectului, echipele **pot utiliza instrumente bazate pe inteligență artificială**, precum ChatGPT, Claude, Gemini sau alte instrumente similare.

Pentru fiecare **utilizare semnificativă a instrumentelor AI**, echipa va documenta:

- instrumentul AI utilizat;
- scopul utilizării;
- promptul sau descrierea solicitării adresate instrumentului AI;
- rezultatul obținut;
- modificările efectuate de membrii echipei asupra rezultatului;
- modul în care rezultatul a fost verificat și validat.

La prezentare, echipa trebuie să poată **explica utilizările semnificative ale instrumentelor AI** și să demonstreze că rezultatele generate au fost **analizate, verificate și validate de membrii echipei** înainte de a fi utilizate în cadrul proiectului.

Cerinte proiecte laborator

Sa se scrie un program (in Java), precum si cerintele (specificatia) acestuia.

1. Pe baza cerintelor programului, sa se genereze date de test folosind a) *equivalence partitioning* b) *boundary value analysis* si c) *cause-effect graphing*. Sa se implementeze testele obtinute folosind JUnit (sau similar pentru alt limbaj de programare).
2. Sa se stabileasca nivelul de acoperire realizat de fiecare dintre seturile de teste de la 1) a), b) si c), rulat separat (nu cumulat cu celelalte seturi), folosind unul dintre utilitarele de code coverage prezentate in [6]. Sa se compare si sa se comenteze rezultatele obtinute de cele trei seturi de teste.
3. Sa se transforme programul intr-un graf orientat si sa se calculeze complexitatea ciclomatica a circuitului derivat din acesta. Sa se identifice un set de circuite linear independende care acopera toate arcele grafului.
4. Sa se gaseasca un set de teste care satisface criteriul *modified condition/decision coverage* (MC/DC).
5. Sa se scrie un mutant de ordinul 1 echivalent al programului.
6. Pentru unul dintre cazurile de testare de la punctele 1 sau 4 sa se scrie un mutant ne-echivalent al programului care sa fie omorat de catre test si un mutant ne-echivalent care sa nu fie omorat de catre test.
7. Sa se foloseasca doua mutation tools pentru a genera mutantii programului implementat. Pentru fiecare dintre aceste utilitare, sa se calculeze mutation scores pentru testele de la punctele 1 si 4. Sa se explice eventualele diferente dintre scoruri, inclusiv diferentele care provin din operatorii de mutatie diferiti folositi de cele doua unelte.

Observatii:

- se poate folosi orice limbaj de programare care permite scrierea testelor unitare si folosirea unui tool de acoperire.
- se poate testa o metoda (functie, procedura) din program, insa trebuie mentionat clar acest lucru, descrisa functionalitatea acesteia, iar toate cerintele se vor rezolva pe baza aceleiasi metode.
- pe langa programul efectiv si testele implementate, se va realiza si o documentatie in care se vor descrie detaliat cum s-au obtinut rezultatele pentru fiecare din cerintele proiectului.
- Graful asociat programului va fi realizat cu un utilitar (de exemplu: Lucidchart, Draw.io, yEd, Microsoft Visio).

Bibliografie

[1]. Functional testing, material de curs.
[2]. Structural testing, material de curs.
[3]. Mutation testing, material de curs.
[4]. http://www.vogella.com/tutorials/JUnit/article.html
[5]. https://cs.gmu.edu/~offutt/mujava/
[6]. https://www.softwaretestinghelp.com/code-coverage-tools/
[7]. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=4755ebf92ea2bfcbd603ebf3ab14e852dd64e978
[8]. https://pitest.org/ – PIT (PITest), mutation testing tool pentru Java/JVM.
[9]. https://mutation-testing.org/ – Major, framework de mutation testing pentru Java.